

Joga Ai

Sistema de Empréstimo de Jogos de Tabuleiro

Sprint 3 - Documentação Técnica Completa

Disciplina:	DCC603 - Engenharia de Software I
Instituição:	Universidade Federal de Minas Gerais (UFMG)
Professor:	Eduardo Figueiredo
Tema:	11 - Sistema de Controle de Empréstimo de Jogos
Equipe:	Alvaro Lima, Beatriz Nogueira, Lucas Dolabella, Sarah Menks, Vanessa Nascimento
Entrega:	10/06/2026

Sumario

1. Visao Geral do Sistema
2. Tecnologias Utilizadas
3. Instalacao e Execucao
4. Arquitetura do Sistema
5. Classes OOP - Implementacao do Diagrama de Classes
6. API REST - Endpoints
7. Banco de Dados
8. Frontend - SPA
9. Fluxos Principais
10. Requisitos Implementados
11. Testes de Software
12. Sistemas Externos
13. Consideracoes Finais

1. Visao Geral do Sistema

O Joga Ai e uma plataforma web que conecta pessoas interessadas em emprestar e pegar emprestados jogos fisicos, como jogos de tabuleiro e cartas. Desenvolvido como trabalho pratico da disciplina DCC603 - Engenharia de Software I (UFMG), o sistema segue o processo agil SCRUM com desenvolvimento incremental ao longo de 3 sprints.

O usuario se cadastra, registra seus jogos disponiveis, pesquisa jogos de outros usuarios, negocia prazos via chat interno e gerencia o ciclo completo do emprestimo, incluindo confirmacao de devolucao, aplicacao automatica de penalidades por atraso (R\$ 5,00 por dia) e desbloqueio da conta apos pagamento de multa.

1.1 Obietivos por Sprint

Sprint	Objetivo	Entrega
Sprint 1	Planejamento e especificacao de requisitos (RF/RNF, casos de uso)	01/04/2026
Sprint 2	Arquitetura em camadas, diagrama de classes, diagramas UML	27/04/2026
Sprint 3	Implementacao completa, testes unitarios e de integracao	10/06/2026

1.2 Atores do Sistema

Ator	Tipo	Responsabilidades
Usuario	Principal	Cadastrar, logar, gerenciar jogos, solicitar emprestimos, usar chat
Sistema de E-mail	Externo	Enviar notificacoes de prazo vencido e comprovantes
Sistema de Pagamento	Externo	Processar pagamento de multas por atraso

2. Tecnologias Utilizadas

Componente	Tecnologia	Versao	Justificativa
Backend	Python + Flask	3.x / 3.0.3	OOP nativo, modular, REST simples
ORM	Flask-SQLAlchemy	3.1.1	Mapeamento OOP, migracao facil de BD
Banco de Dados	SQLite	embutido	Zero setup, arquivo unico, ideal para TP
Autenticacao	PyJWT	2.8.0	Tokens stateless, ideal para APIs REST
Seguranca	Werkzeug (bcrypt)	3.0.3	Hash seguro de senhas, embutido no Flask
Testes	pytest	9.x	Equivalente ao JUnit, padrao Python
Frontend	HTML5 + CSS3 + JS	vanilla	SPA sem dependencias, sem build step

2.1 Dependencias (requirements.txt)

```
Flask==3.0.3
Flask-SQLAlchemy==3.1.1
PyJWT==2.8.0
Werkzeug==3.0.3
pytest==9.x           # somente para testes
```

3. Instalação e Execução

3.1 Requisitos

- Python 3.10 ou superior
- pip (gerenciador de pacotes Python)

3.2 Passos para executar

```
# 1. Clonar o repositório
git clone <url-do-repositorio>
cd TP-Engenharia-de-Software-I

# 2. Criar e ativar ambiente virtual
python3 -m venv venv
source venv/bin/activate          # macOS / Linux
# venv\Scripts\activate           # Windows

# 3. Instalar dependências
pip install -r requirements.txt

# 4. Iniciar o servidor
python run.py

# 5. Abrir no navegador
# http://localhost:5000

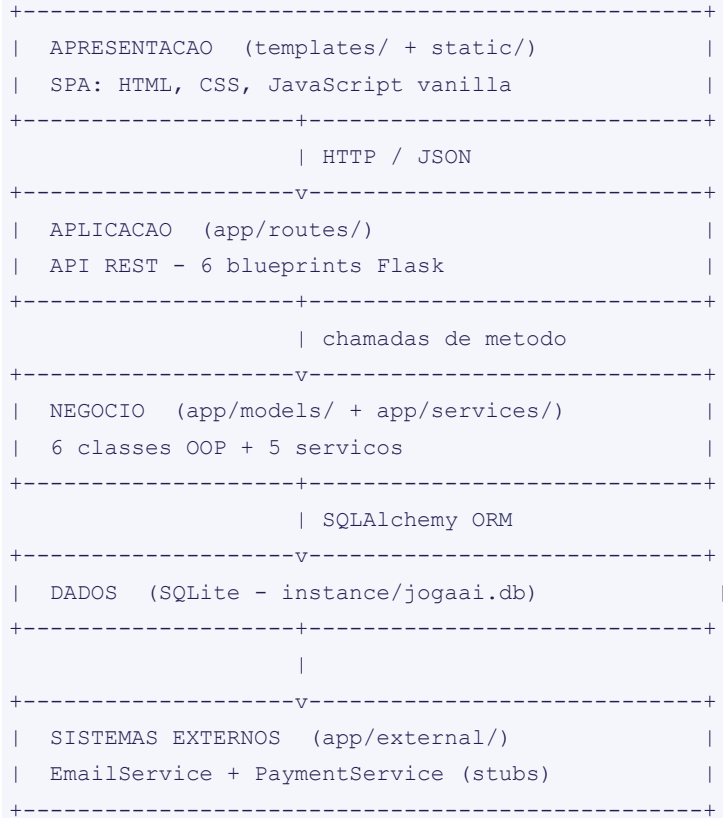
# Para parar o servidor: Ctrl + C
```

O banco de dados SQLite é criado automaticamente em `instance/jogaai.db` na primeira execução. Não é necessário nenhuma configuração adicional de banco de dados.

4. Arquitetura do Sistema

O sistema adota a Arquitetura em Camadas (Layered Architecture), definida na Sprint 2, organizada em cinco camadas hierárquicas com responsabilidades bem separadas. Cada camada só se comunica com a camada imediatamente inferior, garantindo baixo acoplamento.

4.1 Diagrama de Camadas



4.2 Camadas e Correspondencia com o Codigo

Camada	Diretorio	Responsabilidade
Apresentacao	templates/ + static/	Frontend SPA: HTML, CSS, JavaScript
Aplicacao	app/routes/	API REST: recebe req., valida, repassa
Negocio	app/models/ + services/	Classes OOP + regras de negocio
Dados	app/database.py + SQLite	Persistencia via SQLAlchemy ORM
Ext. Sistemas	app/external/	E-mail e pagamento (stubs integraveis)

4.3 Estrutura de Diretorios

```

TP-Engenharia-de-Software-I/
|- run.py                # Ponto de entrada
|- requirements.txt      # Flask, SQLAlchemy, PyJWT, Werkzeug
|- app/
|   |- __init__.py      # App factory (create_app)
|   |- database.py      # Instancia SQLAlchemy
|   |- models/          # Camada de Negocio - Classes OOP
|   |   |- usuario.py   # Usuario + StatusUsuario
|   |   |- jogo.py      # Jogo + StatusJogo
|   |   |- emprestimo.py # Empréstimo + StatusEmpréstimo
|   |   |- mensagem.py  # Mensagem (chat)
|   |   |- notificacao.py # Notificacao + TipoNotificacao
|   |   |- penalidade.py # Penalidade + StatusPenalidade

```

```
| |- services/                # Camada de Aplicacao
| | |- auth_service.py       # Registro, login, JWT
| | |- jogo_service.py       # CRUD de jogos, busca
| | |- emprestimo_service.py  # Solicitar, confirmar, penalizar
| | |- chat_service.py       # Envio e listagem de mensagens
| | |- penalidade_service.py  # Processamento de pagamento
| |- routes/                 # Camada de Apresentacao (API)
| | |- middleware.py         # requer_autenticacao (JWT)
| | |- auth.py, jogos.py, emprestimos.py
| | |- chat.py, penalidades.py, notificacoes.py
| |- external/
|   |- email_service.py      # Stub de e-mail
|   |- payment_service.py    # Stub de pagamento
|- templates/index.html      # SPA principal
|- static/css/style.css
|- static/js/ (api, auth, jogos, emprestimos, chat, app).js
|- tests/
  |- conftest.py             # Fixtures pytest
  |- test_unitario.py        # 35 testes unitarios
  |- test_integracao.py      # 31 testes de integracao
```

5. Classes OOP - Implementacao do Diagrama de Classes

As seis classes principais foram implementadas diretamente a partir do Diagrama de Classes definido na Sprint 2, mantendo atributos, metodos, visibilidades e enumeracoes identicos ao diagrama UML. Cada classe estende db.Model (SQLAlchemy), unindo persistencia e comportamento de negocio no mesmo objeto orientado a objetos.

5.1 Usuario (app/models/usuario.py)

Membro	Tipo / Retorno	Visibilidade	Descricao
id	String (UUID)	publico	Identificador unico gerado automaticamente
nome	String	publico	Nome completo do usuario
email	String (unico)	publico	E-mail usado no login
senha	String	publico	Hash bcrypt da senha
status	StatusUsuario	publico	ATIVO / INATIVO / BLOQUEADO
avaliacao	Float	publico	Media de avaliacoes recebidas
data_cadastro	DateTime	publico	Data de criacao da conta
cadastrar()	void	publico	Inicializa usuario com senha hasheada
login()	bool	publico	Valida e-mail e senha
bloquear()	void	publico	Muda status para BLOQUEADO
desbloquear()	void	publico	Muda status para ATIVO
atualizar_perfil()	void	publico	Atualiza nome ou senha
esta_bloqueado()	bool	publico	Retorna true se status = BLOQUEADO
_validar_senha()	bool	privado	Verifica hash bcrypt da senha

5.2 Jogo (app/models/jogo.py)

Membro	Tipo / Retorno	Visibilidade	Descricao
id	String (UUID)	publico	Identificador unico
nome	String	publico	Nome do jogo
descricao	Text	publico	Descricao completa
categoria	String	publico	Ex: Estrategia, Cartas, RPG
status	StatusJogo	publico	DISPONIVEL / EMPRESTADO
dono_id	FK -> Usuario	publico	Dono do jogo
cadastrar()	void	publico	Valida e inicializa o jogo
verificar_disponibilidade()	bool	publico	True se status = DISPONIVEL
atualizar_status()	void	publico	Muda o status do jogo
buscar(termo)	List[Jogo]	publico	Busca por nome ou categoria (estatico)
listar_por_categoria()	List[Jogo]	publico	Filtra por categoria (estatico)
_validar_dados()	bool	privado	Valida campos obrigatorios

5.3 Empréstimo (app/models/emprestimo.py)

Membro	Tipo / Retorno	Visibilidade	Descrição
id	String (UUID)	publico	Identificador unico
dono_id	FK -> Usuario	publico	Dono do jogo emprestado
solicitante_id	FK -> Usuario	publico	Quem pegou emprestado
jogo_id	FK -> Jogo	publico	Jogo em questao
status	StatusEmprestimo	publico	ATIVO / DEVOLVIDO / CANCELADO
data_inicio	DateTime	publico	Data da solicitacao
data_prazo	DateTime	publico	Prazo acordado para devolucao
data_devolvido	DateTime	publico	Data real de devolucao (nullable)
solicitar()	void	publico	Valida e registra o emprestimo
confirmar_devolucao()	void	publico	Marca DEVOLVIDO e libera o jogo
cancelar()	void	publico	Cancela e libera o jogo
verificar_atraso()	bool	publico	True se agora > data_prazo e ATIVO
calcular_dias_atraso()	int	publico	Dias entre prazo e hoje (0 se no prazo)
atualizar_status()	void	publico	Muda o status do emprestimo
_validar_solicitacao()	bool	privado	Verifica bloqueio, disponib., dono

5.4 Mensagem (app/models/mensagem.py)

Membro	Tipo / Retorno	Visibilidade	Descrição
id	String (UUID)	publico	Identificador unico
remetente_id	FK -> Usuario	publico	Quem enviou
destinatario_id	FK -> Usuario	publico	Quem recebe
emprestimo_id	FK -> Empréstimo	publico	Chat vinculado ao emprestimo
conteudo	Text	publico	Texto da mensagem
lida	Boolean	publico	Se o destinatario ja leu
data_envio	DateTime	publico	Momento do envio
enviar()	void	publico	Valida e prepara a mensagem
marcar_como_lida()	void	publico	Muda lida = True
listar_por_emprestimo()	List[Mensagem]	publico	Historico ordenado por data (estatico)
_validar_conteudo()	bool	privado	Impede mensagem vazia

5.5 Notificacao (app/models/notificacao.py)

Membro	Tipo / Retorno	Visibilidade	Descricao
id	String (UUID)	publico	Identificador unico
destinatario_id	FK -> Usuario	publico	Quem recebe a notificacao
mensagem	Text	publico	Texto gerado automaticamente pelo tipo
tipo	TipoNotificacao	publico	SOLICITACAO_EMPRESTIMO / PRAZO_VENCIDO /
			DEVOLUCAO_CONFIRMADA / NOVA_MENSAGEM /
			PENALIDADE_APLICADA
lida	Boolean	publico	Se o usuario ja visualizou
data_envio	DateTime	publico	Momento do envio
emprestimo_id	FK -> Empréstimo	publico	Empréstimo relacionado (opcional)
enviar()	void	publico	Inicializa e gera a mensagem pelo tipo
marcar_como_lida()	void	publico	Muda lida = True
listar_por_usuario()	List	publico	Retorna notificacoes do usuario (estatico)
_gerar_mensagem()	str	privado	Texto descritivo por tipo de notificacao

5.6 Penalidade (app/models/penalidade.py)

Membro	Tipo / Retorno	Visibilidade	Descricao
id	String (UUID)	publico	Identificador unico
usuario_id	FK -> Usuario	publico	Quem recebeu a penalidade
emprestimo_id	FK -> Empréstimo	publico	Empréstimo em atraso
valor_multa	Float	publico	R\$ 5,00 * dias_atraso (minimo R\$ 5)
status	StatusPenalidade	publico	PENDENTE / PAGA / CANCELADA
data_aplicacao	DateTime	publico	Quando a penalidade foi gerada
data_pagamento	DateTime	publico	Quando foi quitada (nullable)
aplicar()	void	publico	Calcula multa e bloqueia o usuario
confirmar_pagamento()	void	publico	Marca PAGA e desbloqueia o usuario
desbloquear_usuario()	void	publico	Chama usuario.desbloquear()
listar_por_usuario()	List	publico	Penalidades do usuario (estatico)
_calcular_multa()	float	privado	dias_atraso x R\$5 (minimo R\$5)
_bloquear_usuario()	void	privado	Chama usuario.bloquear()

6. API REST - Endpoints

Todas as rotas marcadas como 'Sim' em autenticacao exigem o header:

Authorization: Bearer <token_jwt>

6.1 Autenticacao /api/auth

Metodo	Rota	Auth	Descricao
POST	/api/auth/cadastrar	Nao	Cria conta, retorna JWT + dados do usuario
POST	/api/auth/login	Nao	Login com e-mail e senha, retorna JWT
GET	/api/auth/perfil	Sim	Retorna dados do usuario autenticado
PUT	/api/auth/perfil	Sim	Atualiza nome ou senha

6.2 Jogos /api/jogos

Metodo	Rota	Auth	Descricao
GET	/api/jogos	Nao	Lista jogos (aceita ?busca=termo para filtrar)
GET	/api/jogos/meus	Sim	Lista jogos do usuario autenticado
GET	/api/jogos/:id	Nao	Detalhe de um jogo especifico
POST	/api/jogos	Sim	Cadastra novo jogo
PUT	/api/jogos/:id	Sim	Atualiza jogo (somente o dono)
DELETE	/api/jogos/:id	Sim	Remove jogo (somente dono, se disponivel)

6.3 Emprestimos /api/emprestimos

Metodo	Rota	Auth	Descricao
GET	/api/emprestimos	Sim	Lista emprestimos do usuario (dono ou solicitante)
GET	/api/emprestimos/:id	Sim	Detalhe de um emprestimo
POST	/api/emprestimos	Sim	Solicita emprestimo (jogo_id, data_prazo)
PUT	/api/emprestimos/:id/confirmar-devolucao	Sim	Dono confirma que jogo foi devolvido
PUT	/api/emprestimos/:id/nao-devolvido	Sim	Dono reporta nao devolucao -> penalidade
PUT	/api/emprestimos/:id/cancelar	Sim	Cancela emprestimo ativo

6.4 Chat, Penalidades e Notificacoes

Metodo	Rota	Auth	Descricao
GET	/api/chat/:id	Sim	Lista mensagens (marca nao lidas como lidas)
POST	/api/chat/:id	Sim	Envia mensagem no chat
GET	/api/penalidades	Sim	Lista penalidades do usuario
POST	/api/penalidades/:id/pagar	Sim	Paga multa e desbloqueia a conta
GET	/api/notificacoes	Sim	Lista notificacoes do usuario
PUT	/api/notificacoes/:id/lida	Sim	Marca notificacao como lida
PUT	/api/notificacoes/marcar-todas-lidas	Sim	Marca todas as notificacoes como lidas

6.5 Exemplos de Requisição

Cadastrar usuário:

```
curl -X POST http://localhost:5000/api/auth/cadastrar \
  -H "Content-Type: application/json" \
  -d '{"nome": "Lucas", "email": "lucas@email.com", "senha": "minhasenha"}'
```

Cadastrar jogo (com token):

```
curl -X POST http://localhost:5000/api/jogos \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"nome": "Catan", "descricao": "Jogo de estrategia", "categoria": "Estrategia"}'
```

Solicitar empréstimo:

```
curl -X POST http://localhost:5000/api/emprestimos \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"jogo_id": "<id>", "data_prazo": "2026-06-20T23:59:59"}'
```

7. Banco de Dados

O banco de dados é o SQLite, armazenado em `instance/jogaai.db` e criado automaticamente pelo SQLAlchemy (`db.create_all()`) na primeira execução. O schema reflete diretamente as 6 classes do diagrama de classes.

7.1 Schema das Tabelas

Tabela	Colunas principais	Relacionamentos
usuarios	id, nome, email, senha, status, avaliacao, data_cadastro	1:N jogos, empréstimos, penalidades
jogos	id, nome, descricao, categoria, status, dono_id, data_cadastro	N:1 usuarios, 1:N empréstimos
emprestimos	id, dono_id, solicitante_id, jogo_id, status, datas	N:1 usuarios/jogos, 1:N mensagens
mensagens	id, remetente_id, destinatario_id, emprestimo_id, conteudo, lida	N:1 usuarios/empréstimos
notificacoes	id, destinatario_id, mensagem, tipo, lida, emprestimo_id	N:1 usuarios/empréstimos
penalidades	id, usuario_id, emprestimo_id, valor_multa, status, datas	N:1 usuarios/empréstimos

7.2 Visualizar os Dados

```
# Instalar DB Browser for SQLite (macOS)
brew install db-browser-for-sqlite
open -a 'DB Browser for SQLite'
# Abrir: instance/jogaai.db

# Ou via terminal
sqlite3 instance/jogaai.db
.tables
SELECT * FROM usuarios;
SELECT * FROM empréstimos WHERE status = 'ATIVO';
SELECT * FROM penalidades WHERE status = 'PENDENTE';
.quit
```

8. Frontend - SPA (Single Page Application)

O frontend é uma SPA construída com HTML, CSS e JavaScript puro (vanilla), sem frameworks externos. Todas as telas são renderizadas dinamicamente por JavaScript no mesmo arquivo `index.html`. O roteamento é feito pelo `app.js` que mostra/oculta as views conforme a navegação.

8.1 Telas Implementadas

Tela	Arquivo JS	Funcionalidades
Login / Cadastro	<code>auth.js</code>	Formulários, validação, armazena JWT no localStorage
Explorar Jogos	<code>jogos.js</code> (Jogos)	Grid de jogos, busca em tempo real (debounce 300ms)
Meus Jogos	<code>jogos.js</code> (MeusJogos)	CRUD: adicionar, editar, remover jogos próprios
Empréstimos	<code>emprestimos.js</code>	Lista ativa, histórico, ações por papel (dono/sol.)
Chat	<code>chat.js</code>	Mensagens com auto-refresh a cada 4 segundos
Penalidades	<code>emprestimos.js</code>	Visualizar multas pendentes, pagar com um clique
Notificações	<code>emprestimos.js</code>	Centro de notificações, badge com contador no navbar

8.2 Fluxo de Autenticação no Frontend

Após login/cadastro bem-sucedido, o token JWT é salvo no localStorage do navegador. O objeto Api (`api.js`) injeta automaticamente o header `Authorization: Bearer <token>` em todas as requisições autenticadas. No logout, o token e os dados do usuário são removidos do localStorage.

9. Fluxos Principais

9.1 Fluxo de Empréstimo (Cenário 1 - Sprint 2)

Passo	Ator	Ação do Sistema
1	Solicitante	Busca e seleciona jogo disponível na tela Explorar
2	Solicitante	Clica em 'Pegar Empréstado' e define prazo de devolução
3	Sistema	Valida: usuário não bloqueado, jogo disponível, não é o próprio dono
4	Sistema	Cria Empréstimo (ATIVO), muda Jogo para EMPRESTADO
5	Sistema	Cria Notificação SOLICITACAO_EMPRESTIMO para o dono
6	Ambos	Trocam mensagens via Chat (auto-refresh a cada 4s)
7	Sistema	No vencimento: EmailService.enviar_aviso_prazo_vencido()
8a	Dono	Confirma devolução -> Empréstimo DEVOLVIDO, Jogo DISPONIVEL
8b	Dono	Ou reporta não devolução -> Penalidade criada, solicitante BLOQUEADO

9.2 Fluxo de Penalidade e Pagamento (Cenários 2 e 4 - Sprint 2)

Passo	Ator	Ação do Sistema
1	Dono	Clica em 'Não Devolvido' no empréstimo atrasado
2	Sistema	Penalidade._calcular_multa(): dias_atraso x R\$5,00 (min R\$5)
3	Sistema	Penalidade criada (PENDENTE), usuario.bloquear() chamado
4	Sistema	Notificação PENALIDADE_APLICADA enviada ao solicitante
5	Solicitante	Acessa 'Penalidades', vê valor e motivo da multa
6	Solicitante	Clica em 'Pagar Multa' (seleciona método de pagamento)
7	Sistema	PaymentService.processar(valor, metodo) -> aprovado
8	Sistema	Penalidade.confirmar_pagamento(): status PAGA, data registrada
9	Sistema	usuario.desbloquear(): status ATIVO
10	Sistema	EmailService.enviar_comprovante_pagamento() chamado

9.3 Fluxo de Chat (Cenário 5 - Sprint 2)

Passo	Ator	Ação do Sistema
1	Solicitante/Dono	Clica em 'Chat' no empréstimo ATIVO na tela de Empréstimos
2	Sistema	Verifica se usuário é dono ou solicitante do empréstimo
3	Sistema	Abre interface de chat com histórico carregado
4	Usuário	Digita e envia mensagem
5	Sistema	Mensagem validada (não vazia), salva no BD
6	Sistema	Cria Notificação NOVA_MENSAGEM para o destinatário
7	Destinatário	Recebe mensagem via auto-refresh (polling 4s)

10. Requisitos Implementados

10.1 Requisitos Funcionais

RF	Descricao	Prioridade	Status
RF01	Cadastro de usuarios (nome, e-mail, senha)	Alta	Implementado
RF02	Login / Logout com autenticao JWT	Alta	Implementado
RF03	Cadastrar jogos disponiveis para emprestimo	Alta	Implementado
RF04	Visualizar jogos disponiveis na plataforma	Alta	Implementado
RF05	Busca de jogos por nome ou categoria	Alta	Implementado
RF06	Solicitar emprestimo de um jogo disponivel	Alta	Implementado
RF07	Definir prazo de devolucao no momento da solicit.	Alta	Implementado
RF08	Registrar todas as informacoes do emprestimo	Alta	Implementado
RF09	Chat interno vinculado ao emprestimo	Baixa	Implementado
RF10	E-mail ao dono no vencimento do prazo	Baixa	Stub
RF11	Dono do jogo confirma devolucao	Alta	Implementado
RF12	Bloquear usuario inadimplente	Alta	Implementado
RF13	Aplicar multa por atraso (R\$ 5,00/dia)	Media	Implementado
RF14	Desbloquear conta apos pagamento da multa	Media	Implementado
RF17	Indicar status do jogo (disponivel/emprestado)	Alta	Implementado
RF18	Cancelar solicitacao de emprestimo	Media	Implementado
RF19	Notificacoes internas no sistema	Baixa	Implementado

10.2 Requisitos Nao Funcionais Atendidos

RNF	Descricao	Como foi atendido
RNF01	Facilidade de uso	SPA intuitiva, sem treinamento previo
RNF05	Protecao de dados	JWT + bcrypt, rotas protegidas
RNF06	Criptografia de senha	Werkzeug generate_password_hash (bcrypt)
RNF07	Autenticacao restrita	Middleware requer_autenticacao em todas as rotas
RNF08	Integridade dos dados	Transacoes SQLAlchemy + FK constraints
RNF09	Consistencia de estados	Jogo nao pode ser DISPONIVEL e EMPRESTADO simultaneamente
RNF12	Manutenibilidade	Arquitetura em camadas, modulos independentes
RNF13	Acesso multiplataforma	Aplicacao web, qualquer navegador moderno

11. Testes de Software

O projeto possui 66 testes automatizados divididos em duas suites: testes unitarios e testes de integracao. Os testes usam pytest (equivalente ao JUnit para Python) e um banco SQLite em memoria (isolado por teste), garantindo que cada teste comece com um banco limpo e sem efeitos colaterais entre eles.

11.1 Como Executar os Testes

```
# Ativar o ambiente virtual
source venv/bin/activate

# Executar todos os testes
python -m pytest tests/ -v

# Executar apenas testes unitarios
python -m pytest tests/test_unitario.py -v

# Executar apenas testes de integracao
python -m pytest tests/test_integracao.py -v

# Resultado esperado: 66 passed
```

11.2 Configuracao dos Testes (conftest.py)

O arquivo tests/conftest.py define as fixtures compartilhadas entre todos os testes:

Fixture	Escopo	O que fornece
app	function	Instancia Flask configurada com banco SQLite em memoria
client	function	Flask test client para simular requisicoes HTTP
db	function	Instancia SQLAlchemy com tabelas criadas do zero

A cada teste, o banco e criado do zero (db.create_all()) e destruido ao final (db.drop_all()), garantindo isolamento total entre os testes.

11.3 Testes Unitarios (tests/test_unitario.py) - 35 testes

Testam as regras de negocio das classes isoladamente, sem HTTP nem banco em disco. Cada metodo de negocio e testado de forma independente.

TestUsuario (10 testes)

Teste	O que verifica
test_cadastrar_define_status_ativo	Status inicial do usuario e ATIVO apos cadastro
test_senha_e_armazenada_como_hash	Senha nao e armazenada em texto plano
test_validar_senha_correta	_validar_senha() retorna True para senha correta
test_validar_senha_incorreta	_validar_senha() retorna False para senha errada
test_bloquear_muda_status	bloquear() muda status para BLOQUEADO
test_desbloquear_muda_status	desbloquear() muda status de volta para ATIVO
test_esta_bloqueado_retorna_true	esta_bloqueado() retorna True quando BLOQUEADO
test_esta_bloqueado_retorna_false_quando_ativo	esta_bloqueado() retorna False quando ATIVO
test_atualizar_perfil_nome	atualizar_perfil() modifica o nome corretamente
test_to_dict_nao_expoe_senha	to_dict() nao inclui o campo 'senha'

TestJogo (6 testes)

Teste	O que verifica
test_cadastrar_define_status_disponivel	Status inicial do jogo e DISPONIVEL
test_cadastrar_campos_obrigatorios_vazios_levanta_erro	ValueError para campos vazios
test_verificar_disponibilidade_true	True quando status = DISPONIVEL
test_verificar_disponibilidade_false_quando_emprestado	False quando status = EMPRESTADO
test_atualizar_nome_e_categoria	atualizar() modifica os campos corretamente
test_to_dict_contem_campos_basicos	to_dict() contem nome e status

TestEmprestimo (9 testes)

Teste	O que verifica
test_solicitar_cria_emprestimo_ativo	Status inicial do emprestimo e ATIVO
test_solicitar_usuario_bloqueado_levanta_erro	ValueError se solicitante estiver bloqueado
test_solicitar_jogo_indisponivel_levanta_erro	ValueError se jogo nao estiver disponivel
test_solicitar proprio_jogo_levanta_erro	ValueError se usuario tenta pegar o proprio jogo
test_confirmar_devolucao_muda_status	Status muda para DEVOLVIDO e jogo fica DISPONIVEL
test_cancelar_libera_jogo	Status muda para CANCELADO e jogo fica DISPONIVEL
test_verificar_atraso_false_dentro_do_prazo	False quando data_prazo esta no futuro
test_verificar_atraso_true_prazo_vencido	True quando data_prazo ja passou
test_calcular_dias_atraso	Retorna numero correto de dias de atraso

TestPenalidade (6 testes)

Teste	O que verifica
test_aplicar_calcula_valor_multa	Valor = dias_atraso x R\$5,00
test_aplicar_bloqueia_usuario	aplicar() chama usuario.bloquear()
test_multa_minima_um_dia	Valor minimo e R\$5,00 (1 dia)
test_confirmar_pagamento_muda_status_para_paga	Status muda para PAGA apos pagamento
test_confirmar_pagamento_desbloqueia_usuario	confirmar_pagamento() chama usuario.desbloquear()
test_penalidade_inicial_esta_pendente	Status inicial da penalidade e PENDENTE

TestMensagem (4 testes)

Teste	O que verifica
test_conteudo_vazio_levanta_erro	ValueError para conteudo vazio ou apenas espacos
test_mensagem_inicia_nao_lida	Campo 'lida' inicia como False
test_marcar_como_lida	marcar_como_lida() muda 'lida' para True
test_to_dict_contem_campos_basicos	to_dict() contem campos essenciais

11.4 Testes de Integracao (tests/test_integracao.py) - 31 testes

Testam os fluxos completos via API REST usando o Flask test client. Cada suite cobre um cenario de uso descrito na Sprint 2.

TestAutenticacao (6 testes)

Teste	O que verifica
test_cadastro_retorna_token	POST /cadastrar retorna 201 e token JWT
test_cadastro_email_duplicado_retorna_409	E-mail duplicado retorna HTTP 409
test_login_credenciais_corretas	POST /login com credenciais validas retorna 200 e token
test_login_senha_errada_retorna_401	Senha errada retorna HTTP 401
test_rota_protegida_sem_token_retorna_401	Acesso sem token retorna HTTP 401
test_perfil_retorna_dados_do_usuario	GET /perfil retorna dados sem expor senha

TestJogos (7 testes)

Teste	O que verifica
test_cadastrar_jogo	POST /jogos cria jogo com status DISPONIVEL
test_listar_jogos_sem_autenticacao	GET /jogos funciona sem autenticacao
test_buscar_jogo_por_nome	?busca= filtra corretamente por nome
test_buscar_jogo_por_categoria	?busca= filtra corretamente por categoria
test_remover_jogo_proprio	DELETE retorna 200 para o dono
test_remover_jogo_alheio_retorna_erro	DELETE retorna 400 para quem nao e o dono
test_meus_jogos_retorna_apenas_do_usuario	GET /meus retorna so jogos do usuario autenticado

TestSolicitarEmprestimo (5 testes)

Teste	O que verifica
test_solicitar_emprestimo_com_sucesso	POST /emprestimos retorna 201 e status ATIVO
test_jogo_fica_indisponivel_apos_emprestimo	Jogo muda para EMPRESTADO apos solicitacao
test_nao_pode_solicitar_proprio_jogo	Retorna 400 ao tentar pegar o proprio jogo
test_nao_pode_solicitar_jogo_indisponivel	Retorna 400 se jogo ja estiver emprestado
test_cancelar_emprestimo_libera_jogo	Cancelar muda status do jogo para DISPONIVEL

TestConfirmarDevolucao (3 testes)

Teste	O que verifica
test_dono_confirma_devolucao	PUT confirmar-devolucao retorna status DEVOLVIDO
test_jogo_disponivel_apos_devolucao	Jogo volta para DISPONIVEL apos confirmacao
test_solicitante_nao_pode_confirmar_devolucao	Apenas o dono pode confirmar - retorna 400

TestAplicarPenalidade (3 testes)

Teste	O que verifica
test_nao_devolucao_bloqueia_solicitante	nao-devolvido muda status do usuario para BLOQUEADO
test_usuario_bloqueado_nao_pode_solicitar	Usuario BLOQUEADO recebe 400 ao solicitar emprestimo
test_penalidade_criada_com_valor_correto	Valor da multa e $\geq$ dias_atraso x R\$5,00

TestPagamentoMulta (3 testes)

Teste	O que verifica
test_pagar_multa_desbloqueia_usuario	Apos pagar, usuario volta para ATIVO
test_pagar_multa_marca_penalidade_como_paga	Status da penalidade muda para PAGA
test_outro_usuario_nao_pode_pagar_multa_alheia	Retorna 400 ao tentar pagar multa de outro usuario

TestChat (4 testes)

Teste	O que verifica
test_enviar_mensagem_no_chat	POST /chat retorna 201 com mensagem criada
test_listar_mensagens_do_chat	GET /chat retorna lista com todas as mensagens
test_mensagem_vazia_retorna_erro	Conteudo vazio ou espacos retorna 400
test_terceiro_nao_pode_acessar_chat	Usuario fora do emprestimo recebe 400

11.5 Resumo dos Resultados

Suíte	Arquivo	Total	Passando	Tempo aprox.
Testes Unitarios	test_unitario.py	35	35	~2s
Testes de Integracao	test_integracao.py	31	31	~7s
TOTAL	-	66	66	~9s

12. Sistemas Externos (Stubs)

Os sistemas externos foram implementados como stubs que simulam o comportamento real e registram as operações no log. Em produção, substituir o corpo dos métodos pela integração real sem alterar nenhuma outra camada do sistema.

12.1 EmailService (app/external/email service.py)

Metodo	Quando e chamado	Integracao sugerida
enviar_aviso_prazo_vencido()	Prazo do empréstimo venceu	SMTP / SendGrid
enviar_confirmacao_devolucao()	Dono confirma devolucao	SMTP / SendGrid
enviar_aviso_penalidade()	Penalidade aplicada ao solicitante	SMTP / SendGrid
enviar_comprovante_pagamento()	Multa paga com sucesso	SMTP / SendGrid

12.2 PaymentService (app/external/payment service.py)

Metodo	Comportamento atual (stub)	Integracao sugerida
processar(valor, metodo)	Sempre retorna status='aprovado'	Stripe / MercadoPago / PagSeguro

Nota: A substituição dos stubs por integrações reais não requer mudanças nas camadas de Negócio, Aplicação ou Apresentação, demonstrando o benefício da Arquitetura em Camadas adotada.

13. Considerações Finais

O sistema Joga Ai foi desenvolvido ao longo de 3 sprints seguindo o processo ágil SCRUM, desde a especificação de requisitos até a implementação completa com testes.

A Arquitetura em Camadas adotada na Sprint 2 foi rigorosamente seguida na implementação, resultando em um sistema modular, de fácil manutenção e com baixo acoplamento entre os componentes.

As 6 classes OOP do Diagrama de Classes (Sprint 2) foram implementadas fielmente, mantendo atributos, métodos, visibilidades e enumerações idênticos ao diagrama UML, demonstrando a correspondência direta entre modelagem e código.

Os 66 testes automatizados cobrem todos os cenários de uso definidos na Sprint 2, incluindo fluxos normais e alternativos, garantindo a correção das regras de negócio e dos endpoints da API REST.

13.1 Possibilidades de Evolução

- Substituição do SQLite por PostgreSQL (apenas uma linha de configuração)
- Integração real com serviço de e-mail (SendGrid, SMTP)
- Integração real com gateway de pagamento (Stripe, MercadoPago)
- Implementação de websockets para chat em tempo real (substituir polling)
- Adição de avaliação de usuários após empréstimos (RF15)
- Histórico completo de empréstimos por usuário (RF16)
- Suporte a imagens dos jogos (upload de foto da capa)

13.2 Equipe

Integrante	Matricula	Responsabilidade Principal
Alvaro Lima Santos	2021067062	Penalidades, notificações por e-mail, testes de validação
Beatriz Nogueira Alvares	2023001603	Autenticação, integração BD, testes de integração
Lucas Dolabella de Castro Lopes	2023001549	Cadastro de jogos, busca, documentação, Scrum Master
Sarah Menks Sperber	2023001824	Controle de prazos, integração frontend/backend

Vanessa Nascimento Silva	2023001816	Sistema de emprestimos, chat, correcao de bugs
--------------------------	------------	--